# When does application-level prefetching pay? A negative result from index maintenance

Andrey Borodin

**Аннотация**

Index maintenance reads every page of an index. Once the order of those reads is known in advance — which is what traversing in physical rather than logical order provides — the reads can be issued several at a time instead of one after another, hiding device latency behind request concurrency. We implemented this for three access methods in PostgreSQL 18 using the streaming read interface.

This paper reports that on ordinary hardware and default settings the change makes no measurable difference, and explains why: the operating system already reads ahead of a sequential scan by sixteen pages, performing the same work by a different route. Six runs of a pair of builds differing by exactly the commit in question give a median of 52.5 seconds on both sides. We then show where the benefit does exist, by removing the operating system's contribution: with readahead disabled, physical-order traversal loses almost all of its advantage over logical order (a factor of 1.09 instead of 3.43), which locates precisely the work that readahead was doing and that an application-level mechanism must do when readahead cannot be relied upon.

The practical conclusion is narrower than the one usually drawn from such work, and we think more useful: application-level prefetching of a known page sequence is not an optimisation of the common case but insurance against configurations where kernel readahead is absent, defeated, or wrong — direct I/O, unusual storage stacks, and the pages whose order cannot be predicted. We report the measurement that failed to show a gain rather than the workload that would have.

## 1 Introduction

Reclaiming space in an index requires reading all of it. If the pages are read in an order known only as the traversal unfolds, each read must complete before the next is issued, and the pass proceeds at one device round trip per page. If the order is known in advance, several reads can be outstanding at once, and the pass proceeds at the device's throughput rather than at its latency.

Maintenance of a generalized search tree acquired a known order when its traversal was changed from logical to physical: pages are visited in increasing block number. That change was made for locality — sequential rather than random access — but it also makes the sequence predictable, which is the precondition for issuing reads ahead of need. PostgreSQL 18 gained a streaming read interface for exactly this, and we applied it to maintenance in three access methods.

This paper reports what happened when we measured it, which is nothing, and why that is the correct outcome rather than an implementation failure. It also reports the measurement that locates the missing gain: it was already being collected by the operating system.

**Contributions.**

1. A measurement showing no difference from application-level prefetching in index maintenance under default settings, with the explanation (Section 3).

2. A decomposition, by varying kernel readahead on one device, of how much of the benefit of ordered traversal is attributable to readahead rather than to locality as such (Section 4).

3. A statement of the conditions under which an application-level mechanism is nevertheless required (Section 5).

Таблица 1: Index cleanup time against readahead. Medians of three runs.

| read_ahead_kb | logical, s | physical, s | ratio |
|---|---|---|---|
| 0 | 57.8 | 52.8 | 1.09 |
| 128 (default) | 20.9 | 6.1 | 3.43 |
| 4096 | 16.7 | 6.2 | 2.69 |

## 2 Background

Maintenance visits every leaf and every internal page, removes entries pointing at deleted rows, and reclaims pages that have become empty. Its cost is

$$P = (L + I)\, c_s + \beta\, c_r, \tag{1}$$

where $L$ and $I$ count leaf and internal pages, $c_s$ and $c_r$ are the costs of a sequential and of a random page read, and $\beta$ counts pages that must be read again because a concurrent insertion moved an unprocessed entry behind the sweep. The first term is what ordered traversal buys; the second is what concurrency costs, and its pages arrive in an order nobody knows in advance.

Prefetching acts on the first term in a way the notation hides. Issuing $d$ reads concurrently does not reduce the number of reads; it replaces $n$ latencies by $n/d$ of them, up to the point where the device's throughput binds. Whether this is worth anything depends entirely on whether the reads were being issued one at a time in the first place — and that is a property of the whole storage stack, not of the database.

## 3 The measurement that shows nothing

We compare two builds differing by exactly the commit that puts maintenance of a generalized search tree onto the streaming read interface. Twenty million points, every fifth row deleted, a buffer cache of 256 MB against an index of some 160 000 pages, the operating system's page cache dropped before each measurement, and the table warmed by a scan beforehand so that the only cold pages are index pages.

Six runs give a median of 52.5 seconds on both sides. The two distributions are not merely close; they are the same distribution, and the individual runs interleave.

The reason is visible once one asks what the operating system was doing. The default readahead on this device is 128 KB, that is sixteen pages: on detecting a sequential access pattern the kernel fetches sixteen pages ahead of the reader. Physical-order traversal produces exactly the pattern that triggers this. The work that the application-level mechanism was written to do had therefore already been done, one layer down, for free, and the second implementation of it adds nothing.

We state this plainly because the temptation in reporting such work is to find the configuration in which the number moves. That configuration exists, and Section 5 describes it; but the honest headline for a change shipped to all users is what happens in the setting all users have.

## 4 Locating the benefit by removing readahead

If the kernel is doing the work, one can measure how much work that is by stopping it. We vary `read_ahead_kb` on one device and re-measure the underlying comparison — logical against physical traversal order — on five million points with the cache dropped and the table warmed.

With readahead disabled, ordered traversal is worth 9 %. With it enabled, 3.43 times. The entire advantage of knowing the page order is therefore realized *through* readahead: the ordering itself buys almost nothing directly, because on this device a random read costs nearly what a sequential one does. The number of buffer-cache misses differs by a factor of two in all three rows and is unaffected by any of this, which confirms that what changes is not how many requests are made but whether they can be merged and anticipated.

Two further observations from the same table. Physical-order traversal is reproducible to 0.2 seconds while logical order is not, since the latter's cost depends on how its scattered requests happen to fall. And a very large readahead helps logical order rather than physical: at 4096 KB the former improves from 20.9 to 16.7 while the latter is unchanged at 6.2, already limited by bandwidth. The measured ratio at 4096 KB is thus formally smaller than at the default, which is a reminder that "more readahead" is not a monotone improvement.

## 5 When the application-level mechanism is nevertheless needed

The negative result above is conditional on the kernel being able to help, and there are several ways it cannot.

**When there is no kernel in the path.** A database configured for direct I/O bypasses the page cache and its readahead entirely. This is not exotic: it is the configuration chosen when the database's own buffer cache is sized to the machine and double buffering is judged wasteful.

**When the pattern is not recognized.** Kernel readahead is a heuristic over observed accesses. It recognizes a forward sequential scan of a file. The $\beta$ term of (1) — pages revisited because concurrent insertions moved entries behind the sweep — is by construction not sequential, and no heuristic can anticipate it. Those pages are read the ordinary way, and an application-level mechanism that knew of them could not help either, since their identity is discovered rather than predicted.

**When the layer below has different information.** Network-attached and virtualized storage may reorder, merge, or throttle requests according to rules the kernel's heuristic does not model. Our own device is of this kind, which is why its random and sequential costs are so close and why Table 1 shows a smaller spread than a rotating disk would.

The correct claim for the change, then, is not that it makes maintenance faster. It is that it makes maintenance's I/O pattern explicit rather than emergent, so that performance no longer depends on a heuristic in another layer inferring what the database already knows. That is worth having, and it is a different claim from a speedup, and conflating the two is how performance work acquires its reputation for unreproducibility.

## 6 Limitations

We measured one storage device, one workload and one access method in detail. The negative result is therefore a statement about a common configuration rather than about all configurations, and Section 5 lists cases we have not measured. In particular we have not measured direct I/O, which is the case where we expect the mechanism to pay and where a positive result would be most informative; we report its absence rather than presenting the argument as complete.

The comparison of Section 4 varies readahead on a single device and therefore emulates only the removal of readahead, not the seek cost of a rotating disk. An independent report on such a disk puts the advantage of ordered traversal at fiftyfold [1], which we cannot reproduce and do not treat as ours.

## 7 Conclusion

Application-level prefetching in index maintenance produced no measurable gain in the configuration our users run, because the operating system was already performing the same anticipation from a pattern the traversal order handed it for free. Disabling readahead shows how much that was worth: the advantage of ordered traversal falls from 3.43 to 1.09, meaning almost all of it was readahead and almost none of it was locality as such on this class of device.

We report this as a result because the alternative — searching for the configuration in which the number moves and presenting that — would describe a system nobody runs. The mechanism's value is not speed in the default setting but independence from a heuristic in another layer, which matters exactly where that heuristic is absent or wrong. Establishing that positively requires measuring those configurations, which we have not done, and we would rather leave the claim incomplete than complete it with the wrong evidence.

## Acknowledgements

Melanie Plageman reviewed and committed the three patches this paper reports, for the B-tree, the generalized search tree and the space-partitioned tree, and the error-code work that made their effects observable. Junwang Zhao and Kirill Reshke reviewed the first of them.

The streaming read interface these patches use is not ours; it is the work of Thomas Munro, Andres Freund and others, and this paper is in part a report on what happens when an existing facility is applied to a case that turns out not to need it.

Errors that remain are the author's.

## Список литературы

[1] Jeff Janes. Re: GiST VACUUM. pgsql-hackers mailing list, March 2019. `https://postgr.es/m/CAMkU%3D1wOqJgjXMfO_R_rUjZ6dvba3x-xeCPBLYQZ1pTV1utLow%40mail.gmail.com`.